Actividad Unidad 3: Consultas SQL Empresariales con Apoyo de IA

Entregables 4

Atributo de la Entrega	Detalle del Proyecto Académico
Materia	Business Intelligence — Unidad 3 (Continuidad de la Actividad 2)
Empresa Asignada	Moda Urbana (Retail omnicanal de ropa, calzado y accesorios por variante)
Motor de Base de Datos	MySQL 8.0 (con sql_mode = ONLY_FULL_GROUP_BY activo)
Alcance del Documento	Entregable 4 (Prompts e Iteraciones IA), Entregable 5 (Consultas SQL Inicial y Final Validada) y Entregable 6 (Depuración del Error 1055)
Volumen de Datos Simulados	1,080 líneas en FACT_VENTAS (1,026 completadas y 54 canceladas [5%]), 20 variantes en DIM_PRODUCTO, 6 sucursales en 3 regiones, 181 días (Q1 y Q2 2026)

1. Punto de Partida: Grano y Reglas de Negocio de Moda Urbana

Antes de redactar el prompt y construir las consultas SQL, se verificó la correspondencia exacta con los entregables 1, 2 y 3 elaborados por el equipo para la empresa Moda Urbana:

> **Declaración Oficial del Grano de la Tabla de Hechos (FACT_VENTAS)**
>
> Una fila de FACT_VENTAS representa una variante de producto (combinación específica de modelo, talla y color identificada por producto_key y sku) vendida dentro de una línea de una orden de compra.
>
> Implicación en conteos y agregaciones: Dado que una orden (orden_id) puede contener múltiples líneas con distintas variantes, el conteo de pedidos requiere COUNT(DISTINCT orden_id), el conteo de clientes requiere COUNT(DISTINCT cliente_key) y la rentabilidad por variante se obtiene sumando SUM(margen_total) agrupado por producto_key.

1.1 Respuestas a las Preguntas de Control Previas a Programar

Pregunta de Negocio / Técnica	Decisión Documentada en el Modelo de Moda Urbana
1. ¿A qué dimensión corresponde la región?	Proviene de DIM_SUCURSAL_VENTA.region (Región Centro, Región Norte y Región Occidente), que identifica la región donde se originó comercialmente la venta. DIM_SUCURSAL_ENTREGA se reserva para logística de Click & Collect.

Pregunta de Negocio / Técnica	Decisión Documentada en el Modelo de Moda Urbana
2. ¿Qué columna representa ventas netas y cuál costo?	FACT_VENTAS.venta_neta almacena el ingreso neto de la línea (precio_lista * cantidad * (1 - descuento_pct)) y FACT_VENTAS.costo_total almacena el costo (costo_unitario * cantidad). La rentabilidad es la columna generada FACT_VENTAS.margen_total = venta_neta - costo_total.
3. ¿Cómo se tratan cancelaciones y devoluciones?	Las ventas canceladas (5% de las filas, 54 líneas) se excluyen mediante el filtro WHERE f.estado_venta = 'completada'. Las devoluciones residen en FACT_DEVOLUCIONES como un proceso separado con grano y fecha propios, por lo que no se cruzan directamente en el JOIN de ventas para no duplicar filas.
4. ¿Qué ocurre si un producto no tiene ventas en Q1 2026?	Se utiliza LEFT JOIN desde el agregado de Q2 hacia el agregado de Q1 por (region, producto_key). Si Q1 es NULL, rentabilidad_q1 se muestra como 0.00 con COALESCE, la variación porcentual queda en NULL (usando NULLIF) y la columna indicadora reporta 'sin_dato_periodo_anterior'.
5. ¿Cómo se comprueba que cada región devuelva máximo 5 productos?	Se aplica la función de ventana ROW_NUMBER() OVER (PARTITION BY q2.region ORDER BY q2.rentabilidad_q2 DESC, q2.producto_key ASC) y se filtra WHERE posicion_region <= 5, evitando que un empate en margen devuelva más de 5 filas por región.

2. Entregable 4: Prompts, Respuesta Inicial de IA y Registro de Decisiones

Siguiendo la plantilla obligatoria de la guía de la Unidad 3, se diseñó un prompt estructurado con los siete elementos requeridos (Contexto, Esquema, Grano, Métricas, Filtros, Salida y Validación) para que el asistente de IA generara una consulta trazable contra el esquema estrella real de Moda Urbana.

2.1 Verificación de Elementos Obligatorios del Prompt

Elemento Obligatorio	Contenido Incluido en el Prompt del Equipo
Contexto	Empresa retail Moda Urbana (ropa, calzado y accesorios), reporte comparativo de rentabilidad trimestral por región en MySQL 8.0.
Esquema	FACT_VENTAS f, DIM_PRODUCTO p, DIM_SUCURSAL_VENTA sv, DIM_TIEMPO t, con sus claves primarias, foráneas y nombres exactos de columnas.
Grano	Una fila en FACT_VENTAS equivale a una variante de producto (talla y color) vendida dentro de una línea de una orden de compra.
Métricas	Rentabilidad = SUM(f.margen_total), donde margen_total es columna generada (venta_neta - costo_total); variación absoluta y variación porcentual.

Elemento Obligatorio	Contenido Incluido en el Prompt del Equipo
Filtros	f.estado_venta = 'completada', t.anio = 2026, comparación de t.trimestre = 2 (actual) vs t.trimestre = 1 (anterior); FACT_DEVOLUCIONES separada.
Salida	Región, posición regional (1 a 5), identificador y nombre/variante del producto, rentabilidad Q2, rentabilidad Q1, variación absoluta y %, e indicador.
Validación	Solicitud explícita de enumerar supuestos previos al SQL, explicar CTEs/JOINs/ventana, evitar duplicados de grano y proteger división entre cero.

2.2 Prompt Inicial Completo Enviado a la IA

Prompt Inicial (Plantilla Oficial Aplicada al Esquema de Moda Urbana)

Actúa como analista SQL para MySQL 8.0.

Trabajamos con la empresa Moda Urbana (retail omnicanal de moda por variante de talla y color) y necesitamos construir un reporte reproducible que identifique los 5 productos (variantes) con mayor rentabilidad en cada región durante el segundo trimestre de 2026 (Q2) y muestre su variación absoluta y porcentual frente al primer trimestre de 2026 (Q1).

El grano de la tabla de hechos FACT_VENTAS es: una fila representa una variante de producto (talla y color) vendida dentro de una línea de una orden de compra.

El esquema disponible (con alias permitidos) es:

- FACT_VENTAS f: linea_id (PK), orden_id, cliente_key (FK), producto_key (FK), sucursal_venta_key (FK), sucursal_entrega_key (FK), canal_key (FK), fecha_key (FK), cantidad, precio_lista_total, descuento_total, venta_neta, costo_total, margen_total (columna GENERATED = venta_neta - costo_total), tipo_precio, estado_venta.

- DIM_PRODUCTO p: producto_key (PK), variante_id, sku, talla, color, nombre, categoria, coleccion, temporada, marca, costo_unitario.

- DIM_SUCURSAL_VENTA sv: sucursal_venta_key (PK), sucursal_id, nombre, ciudad, region, tipo_tienda.

- DIM_TIEMPO t: fecha_key (PK), fecha, anio, mes, trimestre, temporada_comercial.

Las reglas de negocio para calcular rentabilidad y filtrar registros son:

1. La rentabilidad se mide con el margen bruto acumulado: SUM(f.margen_total) equivalente a SUM(f.venta_neta - f.costo_total).

2. La región comercial corresponde a la sucursal donde se originó la venta (DIM_SUCURSAL_VENTA.region).

3. Se deben excluir las ventas canceladas aplicando el filtro f.estado_venta = 'completada'.

4. Las devoluciones están en otra tabla de hechos (FACT_DEVOLUCIONES) con su propio grano y fecha, por lo que no se deben unir a FACT_VENTAS.

5. Si un producto tiene ventas en Q2 2026 pero no tuvo registros en Q1 2026, no debe perderse ni arrojar error

> de división entre cero: debe mostrarse con un indicador 'sin_dato_periodo_anterior'.
>
> La salida debe contener: region, posicion_region (Top 1 a 5), producto_key, sku, nombre_producto, talla, color, rentabilidad_q2, rentabilidad_q1, variacion_absoluta, variacion_porcentual e indicador_periodo_anterior, comparando el trimestre 2 de 2026 (actual) con el trimestre 1 de 2026 (anterior), ordenado por región y posición.
>
> Antes de escribir SQL, enumera tus supuestos. Después presenta la consulta y explica cada CTE, JOIN y función de ventana, detallando cómo evitas duplicar importes y cómo previenes la división entre cero.

2.3 Respuesta Inicial de la IA y Análisis Crítico del Equipo

En su primera respuesta, la IA declaró sus supuestos correctamente respecto a las llaves primarias de las dimensiones (evitando fan-out al unir muchos-a-uno desde FACT_VENTAS hacia DIM_PRODUCTO, DIM_SUCURSAL_VENTA y DIM_TIEMPO). Sin embargo, propuso una consulta con cuatro aspectos que el equipo decidió corregir antes de darla por validada:

- **Uso de RANK() sin desempate determinístico:** La propuesta inicial de la IA empleó RANK() OVER (PARTITION BY region ORDER BY rentabilidad_q2 DESC). Al auditar los datos de Moda Urbana, el equipo detectó un empate exacto en Región Norte entre SKU-0006 (Chamarra Otoño L Azul) y SKU-0019 (Abrigo Largo M Negro), ambos con $5,760.00 en Q2 2026. Si un empate ocurriera en el 5° lugar, RANK() devolvería 6 filas para esa región, incumpliendo la regla de máximo 5 productos por región.
- **Agregación condicional en una sola CTE vs. LEFT JOIN entre trimestres:** La IA agrupó Q1 y Q2 dentro de una misma CTE usando SUM(CASE WHEN t.trimestre = ...). Aunque funciona en SQL general, la ficha de Definiciones de Negocio del equipo (Entregable 3) especifica explícitamente resolver la comparación mediante dos agregados trimestrales unidos con LEFT JOIN para distinguir con claridad la ausencia de filas en Q1 (q1.rentabilidad_q1 IS NULL) frente a un margen igual a cero.
- **Omisión de atributos de variante (sku, talla, color):** La primera versión de la IA solo seleccionó producto_key y nombre, omitiendo sku, talla y color. Dado que en Moda Urbana cada nombre de producto tiene dos variantes en DIM_PRODUCTO (talla M/Negro y talla L/Azul), omitir la talla y el color hacía parecer que el mismo nombre aparecía repetido en el Top 5 (como ocurre con Bota Urbana en todas las regiones).
- **Recálculo manual de (venta_neta - costo_total) e indicador genérico:** La IA calculó SUM(f.venta_neta - f.costo_total) en lugar de aprovechar la columna generada almacenada SUM(f.margen_total) y utilizó la cadena 'Sin datos en Q1' en vez del identificador normado 'sin_dato_periodo_anterior'.

2.4 Prompt de Iteración y Registro de Decisiones sobre la IA

Segundo Prompt (Iteración de Refinamiento Enviada por el Equipo)

Ajusta la consulta anterior para cumplir estrictamente con nuestro documento de Definiciones de Negocio de Moda Urbana:

1. Separa el cálculo en dos CTEs independientes (rent_q2 para trimestre = 2 y rent_q1 para trimestre = 1) y únelas en una tercera CTE mediante LEFT JOIN por (region, producto_key).

2. Incluye las columnas p.sku, p.talla y p.color además de p.producto_key y p.nombre, ya que el grano de DIM_PRODUCTO es por variante.

3. Utiliza directamente SUM(f.margen_total) como medida de rentabilidad.

4. Cambia RANK() por ROW_NUMBER() OVER (PARTITION BY q2.region ORDER BY q2.rentabilidad_q2 DESC, q2.producto_key ASC) para garantizar máximo 5 filas por región y desempate determinístico.

5. Usa exactamente el texto 'sin_dato_periodo_anterior' cuando q1.rentabilidad_q1 IS NULL.

Aspecto Evaluado	Propuesta Inicial de la IA (v1)	Decisión del Equipo (Versión Final)	Evidencia Técnica / Regla Aplicada
Cálculo de rentabilidad	SUM(f.venta_neta - f.costo_total)	Aceptar lógica pero usar SUM(f.margen_total)	Definiciones.pdf y vista_fact_ventas.sql: columna GENERATED STORED.
Estructura de CTEs y cruce de periodos	1 CTE con SUM(CASE WHEN trimestre...) para Q1 y Q2	Corregir a 2 CTEs (rent_q2 y rent_q1) unidas con LEFT JOIN	Definiciones.pdf: 'se resuelve con LEFT JOIN entre los agregados de los dos trimestres'.
Identificación del producto	Solo p.producto_key y p.nombre	Agregar p.sku, p.talla y p.color al SELECT y GROUP BY	README.md y dim_producto.sql: 10 modelos x 2 variantes (M/Negro y L/Azul).
Función de ventana para Top 5	RANK() OVER (PARTITION BY region ORDER BY rentabilidad_q2 DESC)	Reemplazar por ROW_NUMBER() con desempate por p.producto_key ASC	Conteo de control en Región Norte: empate en $5,760.00 entre producto_key 6 y 19.
Manejo de nulos y división entre 0	NULLIF(rentabilidad_q1, 0) y etiqueta 'Sin datos en Q1'	Mantener NULLIF/COALESCE y estandarizar a 'sin_dato_periodo_anterior'	Definiciones.pdf: marcador obligatorio sin_dato_periodo_anterior.
Relación con devoluciones	No incluir FACT_DEVOLUCIONES en el JOIN de ventas	Aceptar sin cambios	Definiciones.pdf: FACT_DEVOLUCIONES tiene grano y fecha propios.